بازبینیِ دقیقِ پروژهٔ btc_agent

تمرکز: پل MT5، ریسک، لات، استاپ/تیک‌پروفیت، ساعات بازار، ربات MQL5

روش کار

- خواندنِ خط‌به‌خطِ: schedule_hours.py، market_hours.py، indicators.py، engine.py، mt5_bridge.py، app.py، news.py، backtest.py (مسیرهای send/robot/autorun)، RobotBridge_FullEA.mq5 و RobotBridgeEA.mq5.
- اجرای واقعیِ کدِ اصلی (بدون تغییر) روی شبیه‌ساز کارگزار (mock_mt5) و اعتبارسنجی چند فرمول با داده‌های نمونه.
- **محدودیت:** کتابخانهٔ رسمی MetaTrader5 فقط روی ویندوز است و ترمینال واقعی در دسترس نبود. هر جا رفتار واقعیِ کارگزار مهم است، به «نیاز به تست روی حساب دمو» علامت خورده است.

۱) باگ‌های تأییدشده با اجرا (کدِ اصلی)

۱-۱ لاتِ ریسکیِ کمتر از حداقل بی‌صدا بزرگ می‌شود (حالت risk_pct)

- **محل:** `_norm_volume` (خط «vol = max(vmin, min(vmax, vol))») و `send_order` که `base_vol` ریسکی را از آن عبور می‌دهد.
- **اجرا:** موجودی ۱۰۰۰ دلار، ریسک ۱٪ = ۱۰ دلار، فاصلهٔ استاپ ۵۰۰ پیپ (۰٫۰۵). لاتِ محاسبه‌شده ۰٫۰۰۲ بود؛ کد آن را بی‌صدا ۰٫۰۱ کرد و ریسکِ واقعی **۵۰ دلار** شد (۵ برابر).
- نکتهٔ دیگر: `_norm_volume` حجمِ صریح را «به نزدیک‌ترین گام» گرد می‌کند؛ مثلاً ۰٫۰۱۵ → ۰٫۰۲ یعنی ریسک ۳۳٪ بیشتر.
- **اصلاح:** `fixes/mt5_bridge_fixes.py` → `_calc_risk_volume` (لاتِ کمتر از حداقل = ردِ سفارش) و `_norm_volume(mode="floor")` برای ریسک.
- **تست اصلاح‌شده:** موجودی ۱۰٬۰۰۰ دلار، ریسک ۱٪، SL = ۱۰ پیپ → حجم ۰٫۹۵ و ریسک ۹۹٫۷۵ دلار (درست).

۱-۲ سفارشِ معلق (execution=pending) موفق هم «ناموفق» ثبت می‌شود → خطرِ سفارشِ تکراری

- **محل:** `send_order` : `ok_leg = r.retcode == TRADE_RETCODE_DONE` (10009).
- سفارشِ معلق در MT5 با **10008 (PLACED)** پذیرفته می‌شود. کد آن را شکست حساب می‌کند؛ `_last_dispatch_dir` ثبت نمی‌شود و در چرخهٔ بعدیِ خودکار (هر ۶۰ ثانیه) **همان سفارشِ معلق دوباره** فرستاده می‌شود. گاردِ پوزیشن فقط پوزیشن‌ها را می‌بیند، نه سفارش‌های معلق، پس تکرار را نمی‌گیرد.

- **اجرا:** با پاسخ 10008، خروجی `ok=False` بود.
- **اصلاح:** `_rc_ok` (10008/10009/10010 موفق) و `pending_orders()` و لحاظ‌کردنِ سفارش‌های معلق در گارد. باید در `_program_position_guard` و `/api/mt5/send` هم اضافه شود؛ در فایل فعلی فقط خودِ `send_order` و `close_all` اصلاح شده‌اند.

۱-۳ استاپِ سمتِ اشتباه بی‌صدا «تصحیح» می‌شود

- **محل:** `_apply_stops_level` (خط‌های `sl = price - min_dist`).
- **اجرا:** خرید با قیمت ۱٫۰۸۵ و SL = ۱٫۰۸۶ (بالای قیمت، سمتِ اشتباه) → کد SL را به ۱٫۰۸۴۹۵ تغییر داد و سفارش باز شد. ریسک با ِSL اولیه (اشتباه) حساب شده بود، پس لات و ریسکِ واقعی با هم نمی‌خوانند و استاپ ممکن است همان لحظه بخورد.
- **اصلاح:** `_validate_stops` → SL/TP سمتِ اشتباه = **ردِ سفارش** با پیامِ روشن؛ فاصلهٔ کمتر از /stops freeze level فقط در جهتِ امن جابه‌جا می‌شود و ریسک از ِSL نهایی محاسبه می‌شود.

۱-۴ استاپ‌لاسِ ساختاری بدون سقف؛ TP3 با R:R کمتر از ۱

- **محل:** `engine.build_trade_plan` → بلوکِ «استاپ: ATR 1.5 + احترام به نزدیک‌ترین سطح»: `sl = min(sl, max(below) - 0.3*atr)` بدون هیچ سقفی (برخلاف استاپ فیبو که سقف ATR×۳ دارد).
- **اجرای واقعی** (کدِ اصلی، EURUSD، ATR = 0.001): ورود ۱٫۰۸۴۷۵، استاپ ۱٫۰۶۹۷ یعنی **۱۵ برابر ATR**. TP1 به ATR ۷٫۵ کشیده شد (`_min_tp1`)، TP3 فقط ۰٫۰۰۹۰ بالاتر بود → **R:R نهایی ≈ ۰٫۶**. یعنی نقشه‌ای که ریاضاً زیان‌ده است ولی به‌صورت «سیگنال خرید» ارسال می‌شود.
- **پیشنهاد اصلاح** (در `engine.py`):

```
MAX_SL_ATR = 3.0
if long:
    below = [s for s in m_sup if s < entry]
    if below:
        cand = max(below) - 0.3 * atr
        if cand >= entry - MAX_SL_ATR * atr:       # فقط اگر در سقف است
            sl = min(sl, cand)
else:
    above = [r for r in m_res if r > entry]
    if above:
        cand = min(above) + 0.3 * atr
        if cand <= entry + MAX_SL_ATR * atr:
            sl = max(sl, cand)
```

- و یک گارد نهایی: اگر `rr` نهایی کمتر از ۱٫۵ (یا ۱٫۰) شد، جهت را `NEUTRAL` کنید یا نقشه را ردّ کنید.

۱-۵ تقسیمِ حجم بین TPها با درصدهای نقشه هم‌خوان نیست

- نقشه `close_pct = [40, 35, 25]` می‌گوید، ولی `_legs` کف می‌گیرد و باقیمانده را به لگِ آخر می‌دهد؛ برای ۰٫۱ لات: ۰٫۰۳ / ۰٫۰۳ / ۰٫۰۴ (یعنی بیشترین حجم روی TP3). `partial` هم با `[0.5, 0.3]` کار می‌کند؛ یعنی سه سیستم درصد متفاوت دارید.
- **اصلاح:** `_legs` در فایل fixes وزن‌های `close_pct` را می‌گیرد (۰٫۱ → ۰٫۰۴ / ۰٫۰۳ / ۰٫۰۳، مجموع دقیق).
- **توصیه:** یک منبع واحد برای درصدها تعریف کنید.

۱-۶ ساعت سرور از «آخرین تیک» گرفته می‌شود؛ آخر هفته ساعت یخ می‌زند

- **محل:** `server_time()` → `dt = fromtimestamp(best)` و آفست از همان تیک.
- تیک‌ها از جمعه شب متوقف‌اند؛ پس ساعت سرور در شنبه/یکشنبه همان جمعه شب می‌ماند و آفست اشتباه (مثلاً -۴۸ ساعت) می‌شود. گیت زمان‌بندی (`_program_gate`) و نمایش ساعت از همین استفاده می‌کنند.
- **اصلاح:** `server_time` در فایل fixes فقط وقتی تیک «تازه» است آفست را به‌روز می‌کند و آن را در `mt5_offset.json` ذخیره می‌کند؛ بدون تیک تازه از آخرین آفست معتبر استفاده می‌شود.

۱-۷ ساعات بازار: DST و تعطیلات نادرست است (`market_hours.py`)

- منطقِ ثابتِ «جمعه ۲۱:۰۰ UTC تا یکشنبه ۲۱:۰۰ UTC» است. در زمستان (ساعت آمریکا برمی‌گردد) بازار فارکس یکشنبه **۲۲:۰۰ UTC** باز می‌شود؛ کد یکشنبه ۲۱:۰۰ را باز حساب می‌کند → ورود در ساعتِ بسته (کارگزار ردّ می‌کند یا اسپرد وحشی).
- وقفهٔ روزانهٔ فلز/کالا/شاخص (۱۷:۰۰–۱۸:۰۰ NY) اصلاً لحاظ نشده است.
- تعطیلات (کریسمس، سال نو، جمعهٔ نیک، بسته‌شدنِ زودهنگامِ تعطیلاتِ آمریکا) نادیده گرفته شده‌اند؛ مثلاً ۲۵ دسامبر به‌عنوان «باز» ثبت می‌شود.
- رول‌اوور ۱۶:۵۵–۱۷:۱۰ NY (اسپردِ منفجر) محافظت نمی‌شود.
- **اصلاح:** `fixes/market_hours.py` (منطقهٔ زمانی America/New_York با DST خودکار، وقفهٔ روزانه، تعطیلات، رول‌اوور، `long_close_within`).
- **تست‌های انجام‌شده:** یکشنبهٔ زمستان 21:30 UTC بسته / 22:05 باز؛ یکشنبهٔ تابستان 21:05 UTC باز؛ وقفهٔ روزانهٔ طلا بسته؛ جمعهٔ نیک بسته؛ ۲۵ دسامبر بسته.

۱-۸ آخر هفته / قبل از تعطیلی: هیچ بستنِ پیش‌دستانه‌ای وجود ندارد

- کد فقط «ورودِ جدید در بازار بسته» را مسدود می‌کند. پوزیشن‌های باز و سفارش‌های معلق از جمعه ساعت ۱۷ NY تا یکشنبه **همان‌طور باز می‌مانند** (ریسک گپِ آخرِ هفته).
- همچنین بستنِ زودهنگامِ تعطیلات وجود ندارد.

- **اصلاح:** `_manage_once` در فایل fixes (`weekend_close_enabled`، `weekend_close_before_min`، `weekend_block_new_min`، `weekend_include_crypto`) و در `send_order` ورودِ جدید را در نزدیکیِ تعطیلی مسدود می‌کند.

۱-۹ ریسک درصدی بر مبنای tick_value (نه ارزشِ زیان) و بدون تبدیلِ ارز

- `trade_tick_value` ارزشِ سود است و برای جفت‌ارزهایی مثل GBPJPY (ارزِ سود = JPY) با حسابِ دلاری عدد را غلط می‌کند؛ `order_calc_profit` ارز را خودِ ترمینال تبدیل می‌کند.
- **اصلاح:** `_loss_per_lot` از `order_calc_profit` استفاده می‌کند و `trade_tick_value_loss` فقط fallback است.

۱-۱۰ مدیریتِ پوزیشن (BE/Trailing/Partial) حالتِ درون‌حافظه دارد

- `_pos_state` و `_partial_plans` در RAMاند. بعد از ری‌استارت پلانِ بستنِ جزئی گم می‌شود و SL فعلی (که شاید از قبل سربه‌سر است) به‌عنوان «ریسک اولیه» خوانده می‌شود → ریسک صفر، تریلینگ از همان لحظه.
- **اصلاح:** وضعیت در `mt5_state.db` (SQLite) ذخیره می‌شود و SL اولیه از تاریخچهٔ سفارشِ بازکننده خوانده می‌شود (`_recover_initial_sl`).

۱-۱۱ ترتیب و گام‌های تریلینگ / بریک‌ایون

- BE با `buffer = ۵ point` بدون احتسابِ اسپرد؛ در کارگزارهایی با اسپردِ بزرگ (طلا/بیت‌کوین) SL روی اسپرد می‌نشیند. **اصلاح:** `breakeven_buffer_mode = spread`.
- تریلینگ با `trailing_points = 300` مطلق است: برای EURUSD ۳۰ پیپ، برای BTC (point = 0.01) فقط ۳ دلار؛ باید برای هر نماد بر پایهٔ R باشد. **اصلاح:** `trailing_mode = "r"` و گامِ حداقلی (`trailing_step_r`) برای جلوگیری از اسپمِ اصلاحِ SL.
- در حالت split، وقتی لگِ TP1 بسته شد بقیه سربه‌سر نمی‌شد. **اصلاح:** `split_be_after_tp1`.

۱-۱۲ جست‌وجوی فازیِ نماد می‌تواند نمادِ اشتباه بدهد

- `resolve_symbol` با سه حرفِ اول (`EUR*`، `XAU*`) نامزدِ اولی را پیدا می‌کرد؛ مثلاً EURUSD ممکن است به EURAUD یا EURGBP برود و XAU به XAUEUR.
- **اصلاح:** فقط «نامِ کامل + پسوندِ کارگزار» (مثلاً EURUSD، EURUSDm، EURUSD.m#)؛ اگر چند گزینه بود، خطا می‌دهد.

۱-۱۳ چکِ dry-run با 10009

- `order_check` در MT5 واقعی موفقیت را با retcode = 0 برمی‌گرداند؛ کد فقط 10009 را موفق می‌داند (dry-run همیشه ناموفق دیده می‌شود). **اصلاح:** `_check_ok` هر دو را می‌پذیرد. (بر اساس مستندات MT5 است و روی ترمینال واقعی تأیید نشده.)

۱-۱۴ سقفِ زیانِ روزانه و «امروز» بر اساسِ ساعت محلیِ VPS

- `_today_stats` با `time.localtime` کار می‌کند؛ اگر منطقهٔ VPS با کارگزار فرق داشته باشد، «روز» جابه‌جا می‌شود. **اصلاح:** روز بر اساس ساعت سرور و سقف زیان روزانه (`daily_loss_limit_pct`) از `history_deals_get` + شناور.

۱-۱۵ محافظِ حجم مارجین

- حجمِ محاسبه‌شده ممکن است مارجینِ بیشتر از موجودیِ آزاد بگیرد. **اصلاح:** `_margin_cap` با `order_calc_margin` و سقف `max_margin_use_pct`.

---

۲) مشکلات رباتِ MQL5 (EA)

۲-۱ نگاشتِ تایم‌فریم در RobotBridge_FullEA.mq5 ناقص است

- `TFfromPeriod` فقط M15/H1/H4/D1/W1 را دارد؛ M1/M5/M10/M30/H2 به `"1h"` تبدیل می‌شوند. یعنی یک EA روی چارت ۵ دقیقه، سیگنالِ ساعتی می‌گیرد و معامله‌اش را با ساعتی تأیید می‌کند.
- **اصلاح:**

```
case PERIOD_M1:  return "1m";
case PERIOD_M5:  return "5m";
case PERIOD_M10: return "10m";
case PERIOD_M15: return "15m";
case PERIOD_M30: return "30m";
case PERIOD_H1:  return "1h";
case PERIOD_H2:  return "2h";
case PERIOD_H4:  return "4h";
case PERIOD_D1:  return "1d";
case PERIOD_W1:  return "1w";
default:         return "";     // از سرور = خالی default_tf استفاده کند
```

- و در `Send` فقط وقتی `tf` غیرخالی است فرستاده شود.

۲-۲ حجم با دو رقم اعشار ارسال می‌شود

- `NumToStr(volume, 2)` : لاتِ ۰٫۰۵ درست می‌رود، اما برای گام‌های ۰٫۰۰۱ (بعضی کارگزارها) گرد می‌شود. **اصلاح:** تعداد رقم را از `SYMBOL_VOLUME_STEP` بگیرید.

۲-۳ پیش‌فرض‌های ایمن و نکتهٔ مستندات

- `InpUseTestEndpoint = true` درست است (معاملهٔ واقعی نمی‌رود)؛ برای معاملهٔ واقعی باید دستی false شود.
- `PipSize` برای نمادهای با digits = 2 یا 3 (مثل XAU) پیپ = 10 × point یا point حساب می‌شود؛ `InpSLPips = 300` در XAU می‌تواند ۳ دلار یا ۳۰ دلار شود. در مستندات EA باید تعریفِ پیپ برای هر نماد نوشته شود.

---

۳) مشکلات منطقی و ریاضیِ دیگر (بدون تست عددیِ کامل)

1. **بک‌تست با موتور زنده همخوان نیست:** بک‌تست فقط TP = 3×ATR تک‌هدف دارد، بدون اسپرد، کمیسیون، لغزش، گپِ آخرِ هفته و بدون بریک‌ایون/تریلینگ. نتیجه‌اش را نمی‌شود معیارِ عملکردِ سیستمِ زنده گرفت.
2. **تحلیلِ عملکرد (analytics)** مدل ۴۰/۳۵/۲۵ را فرض می‌کند که با `_legs` و `partial` اجرا نمی‌شود.
3. **امتیازِ تکنیکال و آستانه:** با مخرجِ ۱۴٫۴ (که با HTF به ۱۶٫۴ می‌رسد) و آستانهٔ ±۱۵، حدود ده شاخص با وزن‌های دلخواه جمع می‌شوند؛ این وزن‌ها هیچ تست بک‌تستی ندارند، پس overfitting قابل بررسی نیست.
4. **repaint (کندل جاری):** `copy_rates_from_pos` از شیفت صفر، کندلِ در حالِ تشکیل را هم می‌دهد. FVG، Liquidity، فیبوناچی و ساختار با `iloc[-1]` روی همین کندل حساب می‌شوند؛ سیگنال در طولِ کندل عوض می‌شود. برای سیگنال باید از کندلِ بسته (شیفت ۱) استفاده شود.
5. **گیتِ زمان‌بندی «ساعات کار»:** بر پایهٔ `server_time` معیوب است (بند ۱-۶).
6. **عدم بررسیِ trade_mode و تازگیِ تیک** قبل از ارسال در مسیر خودکار (در `send_order` فایل fixes اضافه شده).
7. **news_alerts / close_for_news:** پوزیشن‌های مرتبط را می‌بندد ولی سفارش‌های معلقِ همان ارز را نه؛ معلقی که بعد از رویداد پر شود، ریسک خبری دارد.
8. **گاردهای تکراری:** `/api/mt5/send`، `_program_auto_dispatch` و `send_order` هر کدام گاردهای جدا دارند؛ یک منبعِ واحد توصیه می‌شود.

---

۴) فهرست فایل‌های اصلاحی و نحوهٔ استفاده

- **`fixes/mt5_bridge_fixes.py`:** بلوکِ جایگزینِ توابع (به انتهای mt5_bridge.py اضافه شود). شامل کانفیگ‌های جدید، `_norm_volume` (floor/nearest)، `_calc_risk_volume`، `_loss_per_lot`

(order_calc_profit)، `_legs` با وزنِ نقشه، `_validate_stops`، `_order_type` با تبدیلِ معلقِ نزدیک به بازار، `server_time` پایدار، `check_risk_guard` + زیانِ روزانه، `symbol_tradeable`، `_send_with_retry`، `send_order` بازنویسی‌شده، وضعیتِ SQLite، `modify_position` / `close_position` / `cancel_order`، `_manage_once` (بستنِ آخرِ هفته، partial ترتیبی، BE با اسپرد، trailing بر پایهٔ R) و `resolve_symbol` امن.

- **`fixes/market_hours.py`:** جایگزینِ کاملِ market_hours.py (نیاز به `symbols.py` فعلی).
- **تغییراتِ پیشنهادیِ بیرون از این دو فایل:** `engine.py` (بند ۱-۴، سقفِ استاپ)، `RobotBridge_FullEA.mq5` (بند ۲-۱، نگاشتِ تایم‌فریم)، `app.py` (گاردِ سفارشِ معلق در `_program_position_guard` و `/api/mt5/send`).

۵) وضعیتِ تست‌ها

- **تست‌شده روی شبیه‌ساز (کدِ اصلی در برابرِ اصلاح‌شده):** لاتِ ریسکیِ کم، سفارشِ معلق با 10008، SL سمتِ اشتباه، تقسیمِ ۰٫۱ لات بین سه TP، ریسکِ دقیقِ ۱٪.
- **تست‌شده (توابعِ خالص):** DST ساعات بازار فارکس/فلز، وقفهٔ روزانه، تعطیلات، پنجرهٔ «بسته‌شدنِ طولانی».
- **تست‌نشده (نیاز به ترمینالِ واقعی/دمو):** بستنِ جمعه، مدیریتِ پوزیشن بعد از ری‌استارت، `server_time` زنده، `order_calc_profit` و `order_calc_margin` واقعی، رفتارِ retcode 0 در `order_check`.
- **پیشنهاد:** ابتدا روی حسابِ دمو با `execution = market`، `risk_mode = fixed` و حجمِ ۰٫۰۱ یک هفته اجرا شود؛ سپس روی حسابِ واقعی.